# Modèle d'Optimisation de Stratégie pour Compétition Hyrox

Elie

1er février 2026

**Résumé**

This document presents the design, architecture, and mathematical foundation of a simulation model aimed at determining the optimal race strategy for a Hyrox endurance event. By modeling an athlete's energy expenditure through an ordinary differential equation (ODE), the system can test numerous pacing strategies to identify the most effective one while respecting the individual's physiological limits.

## 1 Introduction

The Hyrox competition is a hybrid endurance event that alternates 8 kilometers of running with 8 functional exercise stations. The complexity of the competition lies in managing effort to minimize total time without experiencing premature exhaustion ("hitting the wall"). An inadequate pacing strategy can result in catastrophic performance degradation.

This project proposes a quantitative solution to this problem by modeling the athlete as a dynamic system. The objective is to provide a personalized race plan based on the athlete's physiological data, optimized for overall performance.

## 2 Global Methodology

The complete process is decomposed into three distinct operational phases, orchestrated by a series of Python scripts.

1. **Data Collection :** The athlete records performance data during targeted training sessions. This data, including times, heart rate, and perceived fatigue status, is centralized in a CSV file.
2. **Model Calibration :** The system analyzes collected data to personalize parameters of a physiological model. The energy cost coefficients (`k_...`) are adjusted to reflect the athlete's specific capabilities.
3. **Optimization via Simulation :** Using the calibrated model, the program simulates a large number of race strategies (generated by Latin hypercube sampling). Each simulation calculates the athlete's energy evolution. Non-viable strategies are eliminated, and the fastest among the viable ones is selected.

# 3 The Mathematical Model

The core of the simulator relies on the numerical solution of an ordinary differential equation (ODE) that governs the athlete's state.

## 3.1 Variable Definitions

- $E(t)$ : The athlete's energy level at time $t$, normalized over the interval $[0, 1]$.
- $R$ : The energy recovery rate, a small positive constant.
- $C(t)$ : The baseline energy cost of the activity at instant $t$.
- $F(E)$ : The fatigue factor, a dimensionless function depending on $E$.
- $v(t)$ : The running speed (in m/s) at instant $t$.
- $T_s$ : The total time (in s) to complete exercise station $s$.
- $k_{run}, \alpha_{run}, k_s, k_{rox}$ : Cost coefficients specific to each type of activity.

## 3.2 The Main Differential Equation

Energy evolution is modeled by the following equation, where the change in energy is the difference between recovery and expenditure.

$$\frac{dE}{dt} = R - C(t) \cdot F(E) \tag{1}$$

## 3.3 Energy Cost Modeling $C(t)$

The baseline energy cost is a piecewise function depending on the current activity.

$$C(t) = \begin{cases} k_{run} \cdot v(t)^{\alpha_{run}} & \text{if activity is running} \\ k_{rox} \cdot \frac{100}{T_{rox}} & \text{if activity is a Rox Zone} \\ k_s \cdot \frac{100}{T_s} & \text{if activity is exercise station } s \end{cases} \tag{2}$$

For running, cost increases exponentially with speed. For a Rox Zone or exercise station, the cost per second is inversely proportional to the total time, modeling higher power output for shorter durations.

## 3.4 Fatigue Modeling $F(E)$

The fatigue factor amplifies the effort cost when the athlete is tired (i.e., when $E$ is low). It is modeled by a simple quadratic function.

$$F(E) = 1 + (1 - E)^2 \tag{3}$$

This function ensures that the cost is normal ($F(1) = 1$) when energy is full, and doubled ($F(0) = 2$) when energy is empty.

# 4 The Rox Zone Component

The Rox Zone is a specialized obstacle or functional training segment introduced in modern Hyrox formats. Unlike traditional exercise stations that have fixed difficulty parameters, Rox Zones present an additional layer of complexity in pacing strategy.

## 4.1 Rox Zone Characteristics

- **Variable Difficulty :** Rox Zones can be customized with different intensity levels based on competition format.
- **Time-Dependent Cost :** The energy cost follows the inverse relationship $k_{rox} \cdot \frac{100}{T_{rox}}$, where the athlete's time to complete the zone directly affects energy expenditure.
- **Strategic Importance :** Rox Zones are integrated into the optimization algorithm as distinct segments, allowing the model to evaluate different time allocations across all activities including Rox Zones.
- **Fatigue Amplification :** Like other activities, Rox Zone performance is amplified by the fatigue factor, meaning slower times (higher $T_{rox}$) cost proportionally less energy but increase total race time.

## 4.2 Integration into the Optimization Framework

The optimization algorithm treats Rox Zones as parallel entities to running segments and exercise stations. The strategy vector includes time allocations for each Rox Zone, enabling the optimizer to determine whether spending more time at lower intensity in a Rox Zone yields better overall performance than rushing through it in a fatigued state.

# 5 Computational Architecture

The project is implemented in Python and structured across multiple modules for better organization.

- `main.py` : Entry point that orchestrates the execution of different phases.
- `data/hyrox_calibration_data.csv` : Training data storage file.
- `src/config.py` : Configuration file centralizing all parameters (performance bounds, default parameters, etc.).
- `src/data_loader.py` : Module for loading and validating CSV data.
- `src/model_calibration.py` : Module that adjusts model parameters based on user data.
- `src/simulation_engine.py` : Project core, contains ODE model implementation and solver.
- `src/optimizer.py` : Module responsible for strategy generation and optimal solution identification.

# 6 Usage Procedure

1. **Installation :** Install Python dependencies listed in `requirements.txt` using the command `pip install -r requirements.txt`.

2. **Data Entry :** Fill `data/hyrox_calibration_data.csv` with personal training data, ensuring sessions in both "Fresh" and "Fatigued" states are included.
3. **Configuration :** Adjust performance bounds (`PERFORMANCE_BOUNDS`) in `src/config.py` to reflect your own capability range.
4. **Execution :** Run the main script from the project root with the command `python main.py`.
5. **Analysis :** Analyze the results displayed in the console, which include predicted total time and detailed pacing strategy, as well as the generated graph `hyrox_optimal_strategy.png`.

# 7 Conclusion and Limitations

This model provides a robust framework for optimizing Hyrox race strategy. By translating physiological capabilities into a system of equations, it offers quantitative insights that would be difficult to obtain intuitively.

The main limitations lie in the simplification of the energy model (single energy reservoir) and the basic calibration method. Future improvements could include a multi-compartment model and more sophisticated calibration techniques to account for individual variations in energy metabolism.